APPLICATION PERFORMANCE ENGINEERING

CERTIFICATION
STUDY GUIDE
Day 02

Advanced Trace Analytics & KPIs

Investigate latency and errors at depth with the Trace Explorer, quantify performance through the core KPI families, and turn raw telemetry into dashboards you can defend — with diagrams, hands-on labs, solutions, and review questions.

- Trace Explorer: search & analytics
- Latency Percentiles p50–p99
- Slow Transaction Analysis
- Error Tracking & Exceptions
- Availability & Performance KPIs
- Reliability: MTTD / MTTR / MTBF
- Apdex & UX KPIs
- KPI Dashboards

Prepared for: Vivek Arora
Format: Certification preparation • Full guide • Day 02
Modules: 04–05 • Labs: 5 • Review questions: 22
Version 1.0 — August 2026

Table of Contents

Day 02 builds directly on the Day 01 foundations (spans, traces, the agent pipeline, sampling). Here you move from *collecting* trace data to *analysing* it and expressing system health as measurable KPIs.

Recap & how Day 02 fits

A one-page bridge from Day 01. If any of these are hazy, revisit Day 01 before continuing.

On Day 01 you learned *how trace data is produced and collected*: a request becomes a **trace** (a tree of timed **spans** sharing a `trace_id`); the **Trace Agent** receives, normalises, obfuscates, samples, aggregates, and forwards it; **sampling** (head- vs tail-based) controls cost while **trace metrics are computed on 100% of spans**; and the **service map** visualises dependencies.

Day 02 is about *using* that data. Module 04 shows how to interrogate traces at depth — searching, aggregating, and grouping spans in the **Trace Explorer**, reading **latency percentiles**, isolating **slow transactions**, and analysing **errors and exceptions**. Module 05 turns the raw signals into the **KPIs** the business and SRE teams live by — availability, performance, reliability, and user experience — and shows how to assemble them into a dashboard.

The through-line

Everything in Day 02 is an application of one idea: *a distribution of per-request measurements is richer than any single average.* Percentiles, slow-transaction analysis, error grouping, and Apdex are all ways of reading that distribution to protect the user experience.

Screenshots & live defaults

As on Day 01, UI screenshots appear as labelled figure placeholders — capture them live from your own Datadog account. Retention windows, percentile availability, and defaults evolve; confirm specifics against the docs in Appendix C.

MODULE 04

Advanced Trace Analytics

Interrogate trace data at depth: search and aggregate spans, read latency through percentiles, isolate slow transactions, and turn a flood of errors into a short list of actionable issues.

4.1 The Trace Explorer: search, filter, aggregate, group

The **Trace Explorer** is the primary surface for analysing traces. It lets you query spans, slice them by any tag, and aggregate them into timeseries or top-lists — then export the result to a dashboard, monitor, or notebook. Think of it as a query engine over your spans.

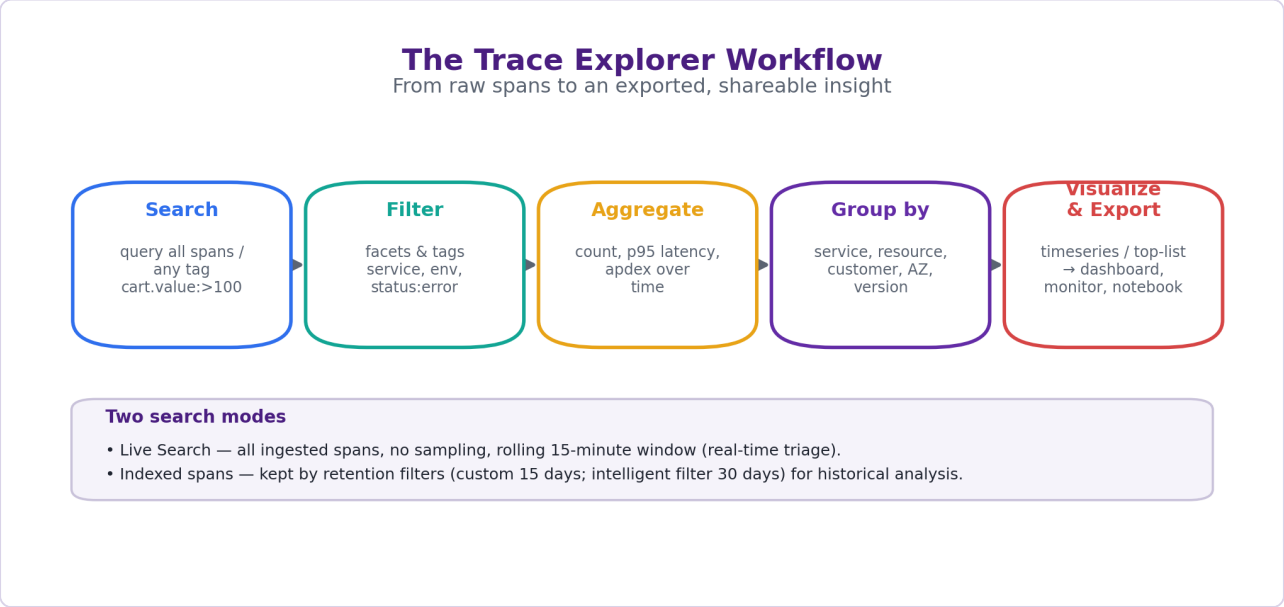

Figure 4.1 — The Trace Explorer workflow: search → filter → aggregate → group by → visualize & export, plus the two search modes (live vs indexed).

Search matches against all spans and any tag — including custom business attributes such as `cart.value` or `customer.tier`. Queries combine free text with tag filters and boolean logic. **Filtering** narrows the set by *facets* (structured, indexed dimensions like `service`, `env`, `resource_name`, `status`) or arbitrary tags; you can also scope by *span type* — all spans, service entry spans, or root spans only.

```
# Example Trace Explorer queries
service:checkout status:error                # errored checkout spans
service:web-store resource_name:"GET /cart" @http.status_code:500
@cart.value:>100 env:prod -customer.tier:internal   # high-value, exclude internal
```

Definition — Facet vs tag; span-level scope

A **tag** is any key–value attached to a span. A **facet** is a tag promoted to a structured, filterable/ aggregatable dimension shown in the sidebar. **Service entry span** = the first span of a service within a trace; **root span** = the first span of the entire trace. Scoping a query to entry or root spans avoids double-counting nested spans when you aggregate.

Aggregation computes a measure over the matched spans — a *count*, or a statistic such as *p95 latency*, error rate, or Apdex. **Group by** then breaks that measure down by one or more dimensions — by `service`, `resource`, `version`, `customer`, or availability zone — producing a timeseries or a top-list. This "aggregate + group by" is exactly what powers trace-based dashboards and monitors.

Operation	What you choose	Example
Search	The span set (text + tags)	`service:checkout status:error`
Aggregate (measure)	count or a statistic	p95 of `duration`
Group by (split)	1–N dimensions	by `resource_name`, top 10
Visualize	timeseries / top-list / table	p95 latency per resource over time
Export	dashboard / monitor / notebook	alert if p95 > 500 ms

Definition — Live Search vs Indexed spans

Live Search queries *all* ingested spans (no sampling bias) within a rolling **15-minute** window — ideal for real-time triage of something happening right now. **Indexed spans** are those retained by **retention filters** (custom filters keep spans for 15 days; the intelligent retention filter keeps a representative set for 30 days) and are what you query for historical analysis. Live = recent & complete; Indexed = historical & filtered.

Exam focus

Two facts are commonly tested: (1) Live Search sees unsampled spans but only for the last 15 minutes; and (2) trace *metrics* (request/error/latency, from Day 01) are computed on 100% of spans regardless of indexing — so aggregate trends stay accurate even for spans you never indexed for search.

4.2 Latency analytics & percentiles (p50–p99)

Latency is not a single number — it is a *distribution*. Averages hide the tail where real users suffer, so APM reports **percentiles**. A percentile pXX is the value below which XX% of requests fall: p95 = 900 ms means 95% of requests were faster than 900 ms and the slowest 5% were worse.

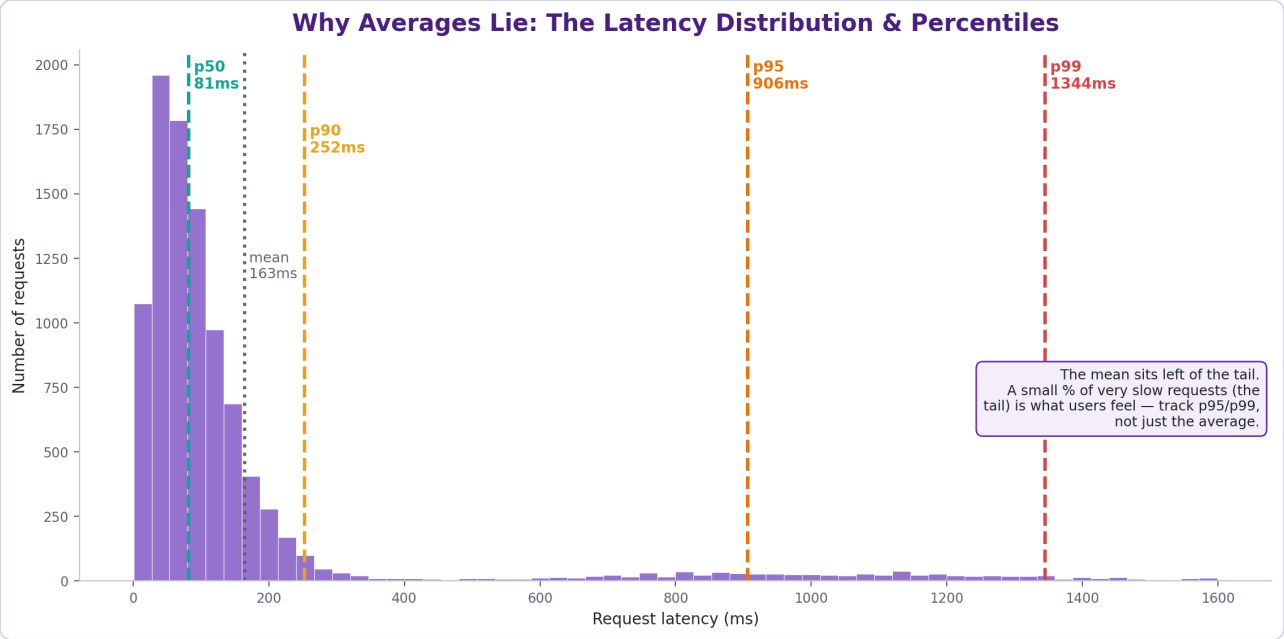

Figure 4.2 — A typical right-skewed latency distribution. The mean sits far left of the tail; p95/p99 capture the experience of the slowest users.

Percentile	Reads as	Use it for
p50 (median)	Typical request	Baseline "normal" experience; robust to outliers.
p75 / p90	Somewhat slow requests	Early-warning of spreading degradation.
p95	Slow tail	Common SLO target; the "most users are at least this fast" line.
p99	Worst 1%	Tail latency, outliers, and the users most likely to churn.

Watch out — averages and un-aggregatable percentiles

Two classic traps: (1) The **mean** is dragged around by outliers and can look "fine" while the p99 is catastrophic — never SLO on the average alone. (2) **Percentiles do not average.** You cannot take the p95 of two services and average them to get a combined p95; percentiles must be computed from the underlying distribution. This is why global percentiles are recomputed from raw data, not derived from per-service percentiles.

Definition — Latency vs duration vs response time

In a span, `duration` is wall-clock time from span start to end. For a request, **response time** (a.k.a. latency) is the root span's duration. "Latency" colloquially covers both; on the exam, treat p50–p99 as percentiles of request response time unless a question scopes it to a specific span.

4.3 Slow transaction analysis

Percentiles tell you a tail exists; **slow transaction analysis** is how you find and explain it. The workflow: filter to the slow set, find the common slow span, and read its context.

1. **Isolate the tail.** In the Trace Explorer, filter the service to `duration:>2s` (or sort traces by duration descending). You now have only the slow requests.
2. **Aggregate & group.** Aggregate p95 latency and group by `resource_name` to see *which endpoint* is slow, then by `version` to check if a recent deploy caused it.
3. **Open the flame graph.** Pick a representative slow trace and read its flame graph — the widest span is the bottleneck (often a DB query, an external HTTP call, or lock contention).
4. **Explain it.** Pivot from that span to its correlated *logs* and, if the span is a DB call, its query and row counts.

> **Tip — Watchdog & suggested facets**
>
> Datadog's **Watchdog** automatically surfaces correlated error/latency tags — e.g. "this latency spike concentrates on `availability-zone:us-east-1b` and `version:1.4.2`." Treat these suggestions as hypotheses to confirm in the flame graph, not conclusions.

4.4 Error analysis, exceptions & failure correlation

At scale you don't have "an error," you have *thousands* of error spans. **Error Tracking** makes this tractable by grouping similar errors into a small number of **issues**.

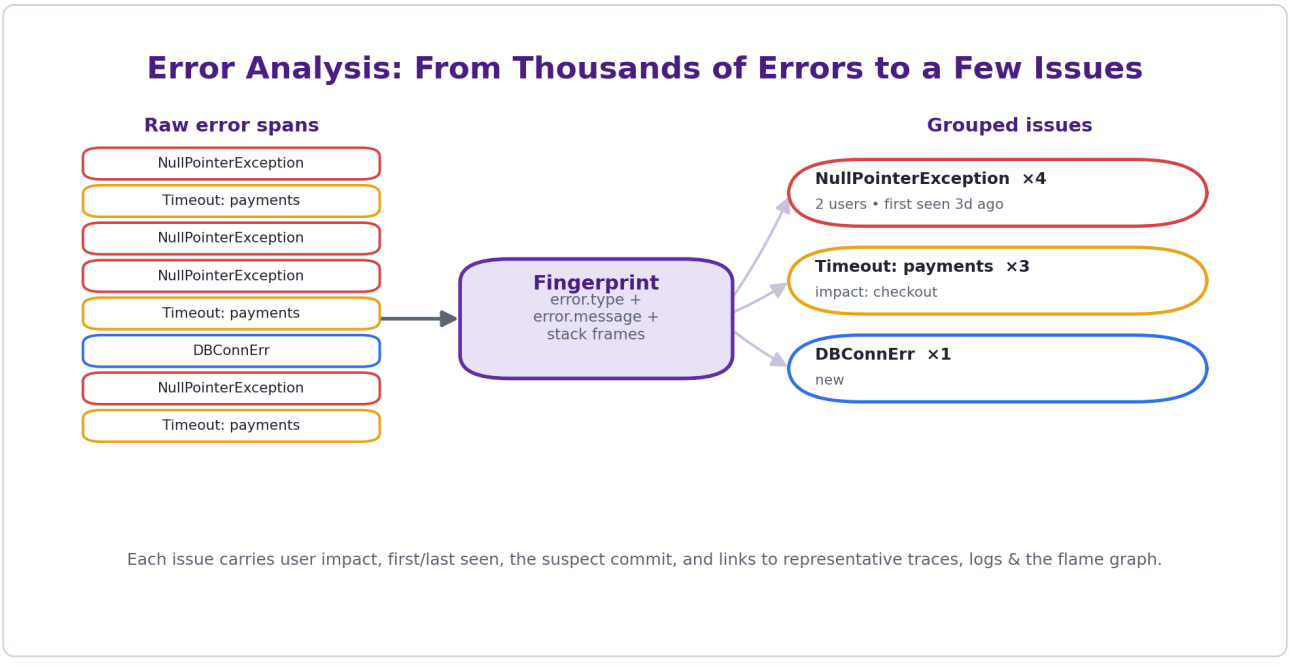

Figure 4.3 — Error Tracking fingerprints each error and groups thousands of error spans into a handful of issues with impact metadata.

Definition — Issue & fingerprint

An **issue** is an aggregation of similar errors — it tells you how many users were impacted, when the error first/last occurred, and often which commit likely caused it. Errors are grouped by a **fingerprint** computed from three attributes: `error.type`, `error.message`, and the **stack trace** frames. Error spans must carry `error.type`, `error.message`, and `error.stack` to be tracked.

Exception analysis drills into a single issue: the exception class, the message, the exact stack frame that threw, the distribution over time, affected hosts/containers/versions, and links to representative traces so you can see the full request context and correlated logs. **Failure correlation** asks "what do the failing requests have in common?" — a specific `version`, endpoint, region, dependency, or customer tier — which is exactly the group-by analysis from §4.1 applied to `status:error`.

Concept	Question it answers	How
Error Tracking	Which distinct problems exist, and which matter most?	Group errors into issues by fingerprint; rank by impact.
Exception analysis	What exactly threw, and where?	Inspect one issue's type/message/stack + representative traces.
Failure correlation	What do failures share?	Group `status:error` by version/resource/region/etc.

Definition — Error rate

The share of requests that failed: `error rate = errored requests ÷ total requests`. In APM it is derived from trace metrics (computed on 100% of spans), so it stays accurate under sampling. Distinguish it from the error *count* — a low-traffic service can have a high error *rate* with a small count, and vice-versa.

Exam focus

Know the three fingerprint inputs (`error.type` + `error.message` + stack frames) and the three required error-span attributes (`error.type`, `error.message`, `error.stack`). Be able to explain why grouping matters: it turns thousands of raw errors into a prioritised, deduplicated worklist with user-impact context.

4.5 Hands-on labs, use cases & review questions

Use case — The "it's fine on average" incident

Support reports intermittent checkout complaints, but the average latency dashboard is flat at ~160 ms. The engineer switches to **p99** and sees it spiking to 1.3 s during traffic peaks. Filtering `service:checkout duration:>1s` and grouping by `resource_name` isolates `POST /checkout`; grouping that slow set by `version` shows the spike began with `version:2.3.0`. The flame graph of one slow trace reveals an N+1 query pattern introduced in that release. *Lesson:* the average hid a tail that percentiles + slow-transaction analysis exposed in minutes.

Lab 4.1 — Trace investigation from symptom to cause

Goal: run the full Trace-Explorer investigation loop.

Users report that "search sometimes hangs." Write the exact sequence of Trace Explorer actions you would take — query, aggregation, group-by, and the final pivot — to locate and explain the cause. State what you expect at each step.

Solution

(1) **Search** `service:search-api` and sort by duration, or filter `duration:>2s` — isolates the slow tail. (2) **Aggregate** p95 `duration`, **group by** `resource_name` — identifies the specific slow endpoint (e.g. `GET /search`). (3) **Group by** `version` and `availability-zone` — checks whether the slowness is deploy- or region-specific. (4) Open a representative slow trace's **flame graph** — the widest span is the bottleneck (say an `elasticsearch` query span). (5) **Pivot** from that span to correlated logs and the query itself — expect evidence such as a missing index or a large result set. Result: symptom → endpoint → scope → bottleneck span → concrete cause.

Lab 4.2 — Slow request analysis with percentiles

Goal: reason correctly about percentiles.

A service shows: mean = 140 ms, p50 = 90 ms, p95 = 800 ms, p99 = 2,400 ms. Answer: (a) What does the gap between p50 and p99 tell you? (b) Your SLO is "p95 < 500 ms" — are you meeting it? (c) Why is the mean (140 ms) lower than p95 but the story still bad? (d) A colleague averages this service's p95 (800) with another's p95 (200) and reports "combined p95 = 500 ms." What's wrong?

Solution

(a) A large p50→p99 gap indicates a **heavy/long tail** — most requests are fast (90 ms) but a minority are dramatically slower (2.4 s), a hallmark of contention, GC pauses, cold caches, or an N+1 pattern hit only sometimes. (b) **No** — p95 = 800 ms exceeds the 500 ms target; the SLO is breached for the slow 5%. (c) The mean is low because the many fast requests dominate the average, but averages ignore that 1–5% of users have a terrible experience — exactly the users who churn. (d) **Percentiles are not additive/averageable**; a combined p95 must be recomputed from both services' underlying distributions, not averaged. The "500 ms" figure is meaningless.

Lab 4.3 — Triage errors into issues

Goal: apply Error Tracking and failure correlation.

Overnight, 12,000 error spans were recorded across `payments`. Describe how Error Tracking reduces this to something actionable, what metadata you would prioritise by, and how you would confirm a single root cause versus several.

Solution

Error Tracking **fingerprints** each error (`error.type` + `error.message` + stack frames) and groups the 12,000 spans into a handful of **issues**. Prioritise by **user impact** and volume (an issue affecting 2,000 users outranks one affecting 3), and by **new vs regressed** issues and the suspect commit. To test single- vs multi-cause: perform **failure correlation** — group `status:error` by `version`, `resource`, `dependency`, and `availability-zone`. If nearly all errors share one dimension (e.g. `version:4.1.0`), that's a single root cause (likely a bad deploy — roll back); if the top issues correlate with different dimensions, treat them as separate problems ranked by impact.

Review questions — Module 04

Q1. What four operations make up the core Trace Explorer workflow, and what does each choose?

Q2. Distinguish Live Search from Indexed spans, including the time windows.

Q3. What is the difference between a tag and a facet?

Q4. Define p95 latency in plain language.

Q5. Give two reasons never to SLO on the mean latency alone.

Q6. Why can't you average two services' p95 values to get a combined p95?

Q7. List the four steps of slow-transaction analysis.

Q8. What three attributes form an error's fingerprint, and what three attributes must an error span carry?

Q9. What is "failure correlation," and which Trace Explorer operation implements it?

Q10. Distinguish error rate from error count with an example.

Answers in Appendix A.

MODULE 05

Advanced Performance KPIs

Express system health as numbers you can target and defend: availability, performance, reliability, and user-experience KPIs — and how to assemble them into a dashboard.

5.1 The four KPI families

A **KPI** (Key Performance Indicator) is a measured signal tied to an objective. Effective observability tracks four complementary families; each answers a different stakeholder question, and a healthy system needs all four.

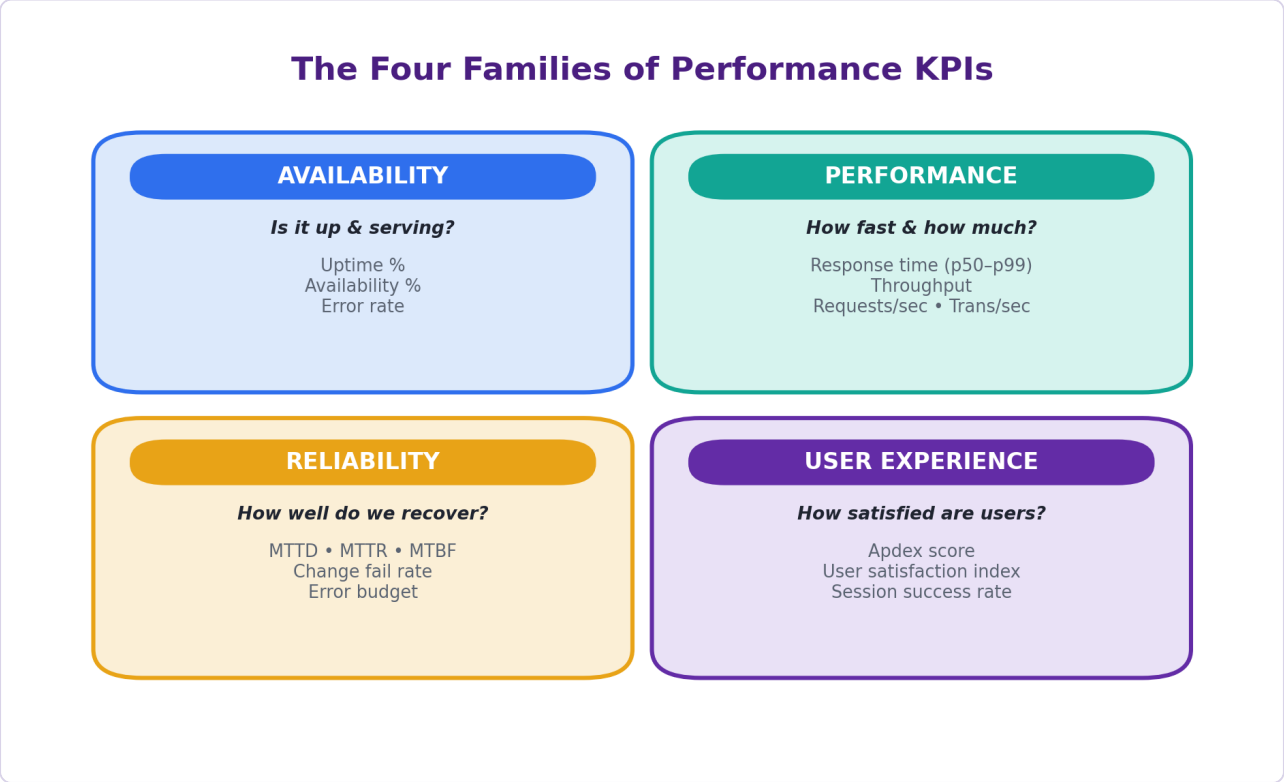

Figure 5.1 — The four KPI families: availability, performance, reliability, and user experience.

Family	Answers	Headline KPIs
Availability	Is it up and serving?	Uptime %, availability %, error rate
Performance	How fast and how much?	Response time (p50–p99), throughput, RPS/TPS
Reliability	How well do we detect & recover?	MTTD, MTTR, MTBF
User experience	Are users satisfied?	Apdex, satisfaction index, session success rate

Tip — Pair a "speed" KPI with a "correctness" KPI

Never present latency without also presenting error rate (and vice-versa). A service can be fast because it's failing fast (returning 500s in 5 ms), or accurate but unbearably slow. The *pair* tells the truth; either one alone can mislead.

5.2 Availability KPIs

Availability KPIs measure whether the service is up and successfully handling requests.

KPI	Definition / formula	Notes
Uptime %	`uptime ÷ (uptime + downtime) × 100`	Time-based; often what SLAs are written against. "Three nines" (99.9%) ≈ 43.8 min downtime/month.
Availability %	`successful requests ÷ total requests × 100`	Request-based (a.k.a. success rate); better reflects user impact than pure uptime for high-throughput services.
Error rate	`errored requests ÷ total requests`	The inverse view of success rate; derived from trace metrics, accurate under sampling.

Definition — The "nines"

Availability targets are shorthanded by nines. 99% ("two nines") ≈ 7.2h downtime/month; 99.9% ("three nines") ≈ 43.8 min; 99.99% ("four nines") ≈ 4.4 min; 99.999% ("five nines") ≈ 26 s. Each extra nine is ~10× harder and costlier — which is why SLOs are chosen deliberately, not maxed out.

Watch out — uptime ≠ availability

A service can be "up" (health check passes) while failing 20% of real requests. Prefer *request-based* availability/success rate for user-facing services; reserve pure uptime for coarse SLAs. On the exam, if a question stresses user impact, the request-based metric is usually the better answer.

5.3 Performance KPIs

Performance KPIs measure speed and capacity.

KPI	Definition	Notes
Response time / latency	Time to serve a request, reported as p50/p75/p90/p95/p99	Always percentiles, never the mean alone (Module 04).
Throughput	Volume of work processed per unit time	The capacity/load side of performance.
Requests per second (RPS)	Inbound requests handled per second	Load indicator; pair with latency to spot saturation.
Transactions per second (TPS)	Completed business transactions per second	Business-level throughput (e.g. orders/s), not just HTTP hits.

Definition — The USE and RED methods

Two standard KPI frameworks worth knowing. **RED** (for request-driven services): *Rate* (RPS), *Errors* (error rate), *Duration* (latency percentiles) — the core APM triad. **USE** (for resources): *Utilization*, *Saturation*, *Errors*. RED describes the service's behaviour; USE describes the resources underneath it.

Tip — Read latency against throughput

Latency rising *as* throughput rises signals saturation (a resource nearing its limit); latency rising while throughput is flat or falling signals a different fault (a slow dependency, GC, lock contention). Plotting the two together is one of the most diagnostic panels on any dashboard.

5.4 Reliability KPIs: MTTD, MTTR, MTBF

Reliability KPIs measure how a team detects and recovers from failures over time. They are best understood on the incident timeline.

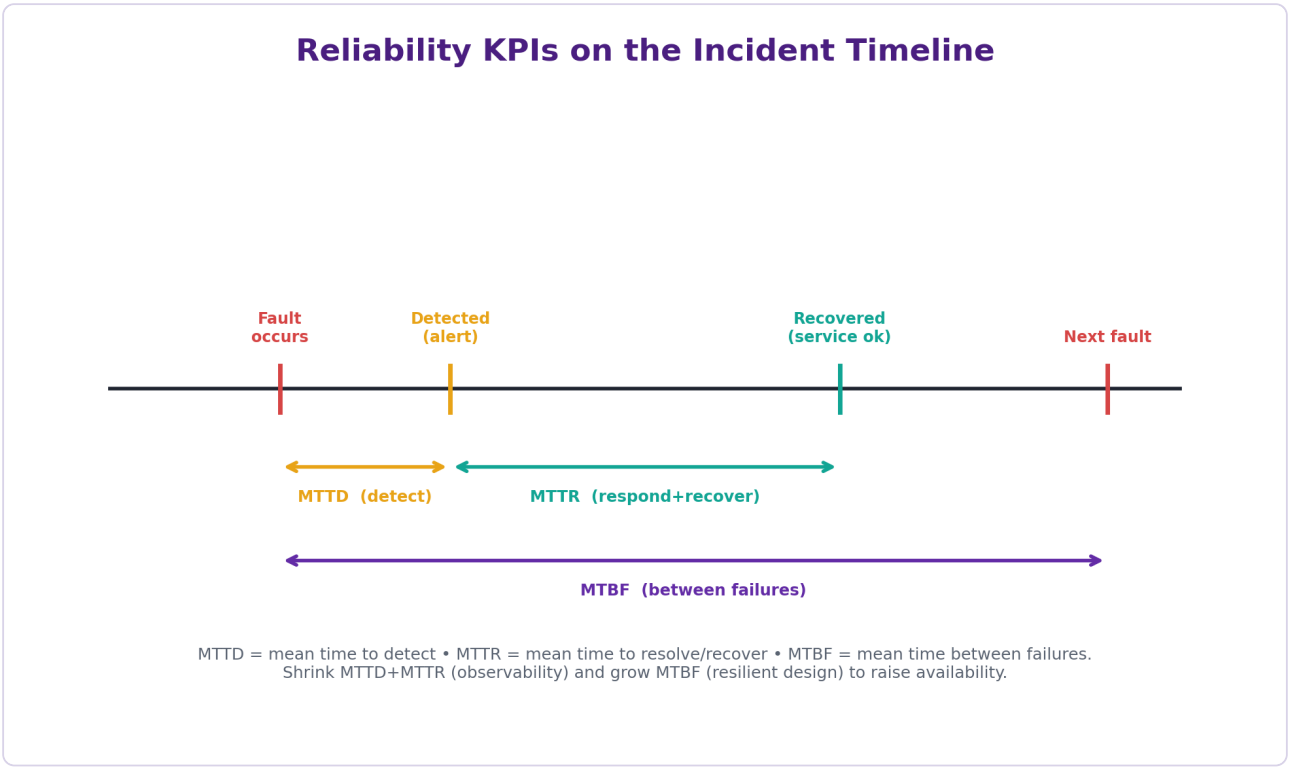

Figure 5.2 — MTTD, MTTR, and MTBF mapped onto an incident's lifecycle.

KPI	Means	Measures	Improve by
MTTD	Mean Time To Detect	Fault occurs → team is alerted	Better monitors, anomaly detection, coverage
MTTR	Mean Time To Resolve / Recover	Detection → service restored	Faster root cause (traces, service map), runbooks, rollback
MTBF	Mean Time Between Failures	Interval between successive failures	Resilient design, fixing root causes, reducing change risk

Definition — MTTA & the MTTR family

You may also meet **MTTA** (Mean Time To Acknowledge — alert fires → a human owns it) and see MTTR expanded as *respond*, *repair*, or *recover*. The exam-safe reading: MTTD then MTTR sum to the user-visible outage; observability primarily shrinks these two, while good engineering grows MTBF.

Exam focus

Remember the timeline order: *fault → (MTTD) → detected → (MTTR) → recovered*, with *MTBF* spanning fault-to-next-fault. Availability improves when you **lower MTTD + MTTR** and **raise MTBF**. Don't confuse MTBF (between failures) with MTTF (mean time to failure, for non-repairable components).

5.5 User-experience KPIs & Apdex

UX KPIs translate raw performance into a satisfaction signal. The best-known is **Apdex** (Application Performance Index), a single 0–1 score.

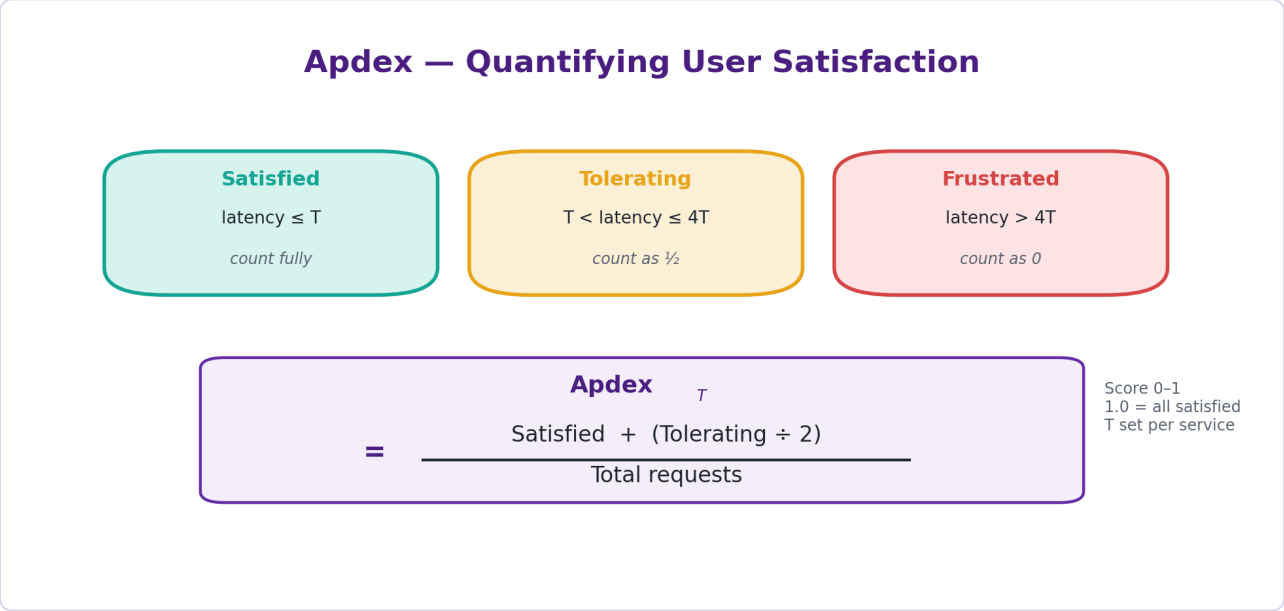

Figure 5.3 — Apdex buckets every request as satisfied, tolerating, or frustrated against a per-service threshold T, then scores 0–1.

Definition — Apdex

Choose a target response-time threshold **T** per service. Each request is **Satisfied** if latency ≤ T, **Tolerating** if T < latency ≤ 4T, or **Frustrated** if latency > 4T. Then:

$$\text{Apdex}_T = (\text{Satisfied} + (\text{Tolerating} \div 2)) \div \text{Total requests}$$

The score runs 0 (all frustrated) to 1 (all satisfied). In Datadog APM, T is **configured per service**, so each service is judged against its own reasonable target.

Worked example. With T = 300 ms and 1,000 requests: 700 ≤ 300 ms (satisfied), 200 between 300 ms and 1,200 ms (tolerating), 100 > 1,200 ms (frustrated). Apdex = (700 + 200/2) ÷ 1000 = **0.80**. Reading: broadly good, but the frustrated 10% is dragging the score — a clear target for slow-transaction analysis.

Other UX KPI	Definition
User Satisfaction Index	A composite of UX signals (latency, errors, and sometimes RUM data) into one trend — a generalisation of the Apdex idea across more inputs.
Session success rate	Share of user sessions completed without a blocking error or abandonment — a journey-level, not request-level, health signal (often from RUM).

Tip — Choosing T

Set T from user expectation for that endpoint, not from current performance — otherwise you "grade on a curve" and a slow service always looks fine. A common starting point is the latency users perceive as responsive for that interaction; then let the frustrated bucket, not vanity, drive improvement.

5.6 Hands-on lab: build a KPI dashboard

Lab 5.1 — Design a service KPI dashboard

Goal: assemble the four KPI families into one coherent, decision-ready dashboard for the `checkout` service.

Specify the panels you would create (one or two per family), the exact measure and grouping for each, and how you would lay them out so an on-call engineer can read health in ten seconds. Then state which two panels you would attach monitors to and why.

Solution

Layout — top row = the RED triad at a glance (biggest, top-left, since it's read first):

1. **Availability:** a stat/query-value tile of request-based success rate (`(total − errors) ÷ total`) with a coloured threshold, plus a small error-rate timeseries beside it.
2. **Performance — latency:** a timeseries of p50, p95, p99 `duration` for `service:checkout`, grouped optionally by `resource_name`; add a second timeseries of RPS/throughput so latency can be read against load.
3. **Reliability:** tiles for MTTD and MTTR (from incident data) and MTBF trend — or, if incident tooling is separate, an SLO widget showing error-budget burn as the practical proxy.
4. **User experience:** an Apdex timeseries (with T annotated) and a session-success-rate tile.

Group/scope: template-variable the dashboard by `env` and `version` so you can compare deploys. **Monitors:** attach alerts to (a) **p95 latency** (user-facing performance SLO) and (b) **error rate / success rate** (availability SLO) — these two catch the majority of user-impacting regressions; Apdex and MTTR are better as trends than as paging alerts because they move more slowly or depend on incident lifecycle. Pair rule honoured: latency and errors are both present so neither misleads.

Use case — Defending an SLO in a review

In a monthly reliability review, a team claims "checkout is healthy — uptime was 99.95%." The dashboard tells a fuller story: request-based **success rate** was 99.2% (uptime hid failed requests on healthy hosts), **p99** breached the 500 ms SLO during two deploys, **Apdex** dipped to 0.82 those days, and **MTTR** was 47 min because the root cause was hard to find. The four families together convert a vague "healthy" into specific, fundable actions: add request-based alerting, gate deploys on p99, and invest in faster root-cause tooling to cut MTTR.

Review questions — Module 05

Q11. Name the four KPI families and one headline KPI for each.

Q12. Why should latency and error rate always be shown together?

Q13. Distinguish uptime % from request-based availability %, and say which better reflects user impact.

Q14. Roughly how much monthly downtime do "three nines" and "four nines" allow?

Q15. Define RPS and TPS and explain the difference.

Q16. State the RED method and the USE method and when each applies.

Q17. Put MTTD, MTTR, and MTBF on the incident timeline and say which two observability most directly reduces.

Q18. Write the Apdex formula and the boundaries of its three buckets.

Q19. Compute Apdex for T = 200 ms given 800 satisfied, 150 tolerating, 50 frustrated (1,000 total).

Q20. Why should the Apdex threshold T be set from user expectation rather than current performance?

Q21. On a KPI dashboard, which two panels are the best candidates for paging monitors, and why?

Q22. Give one reason a service can show high uptime yet poor user experience.

Answers in Appendix A.

Appendix A — Answer keys

Module 04

#	Answer
Q1	Search (choose the span set), Aggregate (choose the measure — count or a statistic like p95), Group by (choose the split dimensions), Visualize/Export (choose timeseries/top-list and send to a dashboard/monitor/notebook).
Q2	Live Search queries all ingested spans without sampling but only for a rolling 15-minute window; Indexed spans are those kept by retention filters (custom 15 days, intelligent filter 30 days) for historical search.
Q3	A tag is any key–value on a span; a facet is a tag promoted to a structured, filterable/aggregatable dimension in the sidebar.
Q4	p95 = the latency value below which 95% of requests fall; only the slowest 5% are worse.
Q5	(1) The mean is skewed by outliers and can look fine while the tail is terrible; (2) it hides the 1–5% of users (p95/p99) who actually suffer and are most likely to churn.
Q6	Percentiles are not additive — a combined p95 must be recomputed from the merged underlying distribution, not averaged from per-service percentiles.
Q7	Isolate the slow tail (filter by duration) → aggregate & group by resource/version → open the flame graph to find the bottleneck span → pivot to logs/query to explain it.
Q8	Fingerprint = `error.type` + `error.message` + stack frames; error spans must carry `error.type`, `error.message`, `error.stack`.
Q9	Finding what failing requests share (version, region, dependency, etc.); implemented by grouping `status:error` spans by those dimensions.
Q10	Error rate is errors ÷ total (a ratio); error count is the absolute number. E.g. 5 errors out of 10 requests = 50% rate but tiny count; 500 out of 5,000,000 = huge count but 0.01% rate.

Module 05

#	Answer
Q11	Availability (uptime/success rate), Performance (latency percentiles / throughput), Reliability (MTTD/MTTR/MTBF), User experience (Apdex / session success rate).
Q12	Either alone misleads: a service can be fast because it fails fast (5 ms 500s), or correct but far too slow. The pair tells the truth.
Q13	Uptime % is time-based (up ÷ total time); availability % is request-based (successful ÷ total requests). Request-based better reflects user impact, since a host can be "up" while failing requests.
Q14	Three nines (99.9%) ≈ 43.8 min/month; four nines (99.99%) ≈ 4.4 min/month.
Q15	RPS = inbound requests/second (technical load); TPS = completed business transactions/second (e.g. orders). TPS is business-level throughput, RPS is request-level.
Q16	RED = Rate, Errors, Duration — for request-driven services; USE = Utilization, Saturation, Errors — for resources. RED describes service behaviour; USE describes the resources under it.
Q17	fault → (MTTD) → detected → (MTTR) → recovered; MTBF spans fault-to-next-fault. Observability most directly reduces MTTD and MTTR.
Q18	$Apdex_T$ = (Satisfied + Tolerating/2) ÷ Total, where Satisfied ≤ T, Tolerating is T–4T, Frustrated > 4T.
Q19	(800 + 150/2) ÷ 1000 = (800 + 75) ÷ 1000 = **0.875**.
Q20	Setting T from current performance grades on a curve — a slow service always "passes." Anchoring T to user expectation keeps the score meaningful and improvement-driving.
Q21	p95 latency (performance SLO) and error rate / success rate (availability SLO): together they catch most user-impacting regressions; Apdex and MTTR are better as trends.
Q22	Uptime only checks the host/health endpoint; the service can still fail or slow real requests (failed transactions, high p99) while reporting "up."

Appendix B — KPI formula & percentile cheat sheet

Formulas

KPI	Formula
Availability (request-based)	successful requests ÷ total requests × 100
Uptime %	uptime ÷ (uptime + downtime) × 100
Error rate	errored requests ÷ total requests
$Apdex_T$	(Satisfied + Tolerating ÷ 2) ÷ Total (Sat ≤ T; Tol T–4T; Frust > 4T)
MTTR	total recovery time ÷ number of incidents
MTBF	total operational time ÷ number of failures
MTTD	total time-to-detect ÷ number of incidents
Throughput / RPS	requests processed ÷ time

Percentiles at a glance

Percentile	Meaning	Typical use
p50	Median / typical request	Baseline normal
p75 / p90	Mildly slow	Early warning
p95	Slow tail	Common latency SLO
p99	Worst 1%	Tail / churn risk

Availability "nines"

Target	Downtime / month (approx.)
99% (two nines)	~7.2 hours
99.9% (three nines)	~43.8 minutes
99.99% (four nines)	~4.4 minutes
99.999% (five nines)	~26 seconds

One-page memory hooks

Explorer: search → filter → aggregate → group by → export; Live = 15 min unsampled, Indexed = 15/30-day retention. **Percentiles:** track p95/p99, never the mean; percentiles don't average. **Errors:** fingerprint = type + message + stack → issues. **KPI families:** Availability, Performance, Reliability, UX. **RED** = Rate/Errors/Duration. **Reliability:** lower MTTD+MTTR, raise MTBF. **Apdex:** (Sat + Tol/2)/Total, T & 4T thresholds, per-service T.

Appendix C — Sources & further reading

Factual claims about the Trace Explorer, Error Tracking, and Apdex are grounded in the official Datadog documentation below. Standard KPI/reliability definitions (percentiles, MTTD/MTTR/MTBF, the nines, RED/USE, Apdex) follow widely-accepted industry usage. Confirm current defaults against the live docs.

- Datadog Docs — Trace Explorer: `docs.datadoghq.com/tracing/trace_explorer/`
- Datadog Docs — Trace queries, facets & aggregation: `docs.datadoghq.com/tracing/trace_explorer/query_syntax/`
- Datadog Docs — Error Tracking (issues & fingerprinting): `docs.datadoghq.com/tracing/error_tracking/`
- Datadog Docs — Configure Apdex by service: `docs.datadoghq.com/tracing/guide/configure_an_apdex_for_your_traces_with_datadog_apm/`
- Datadog Docs — Retention filters & ingestion: `docs.datadoghq.com/tracing/trace_pipeline/`
- Datadog Docs — Dashboards & SLOs: `docs.datadoghq.com/dashboards/` · `docs.datadoghq.com/service_management/service_level_objectives/`
- Apdex specification (Alliance) & the RED/USE methods (industry references).

Day 02 study guide compiled August 2026. Diagrams are original, produced for this guide. UI screenshots are intentionally left as labelled placeholders to capture live from your own Datadog account.